



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2283 - DISEÑO Y ANÁLISIS DE ALGORITMOS

Ayudantía 5 - Programación Dinámica

Septiembre de 2025

Paula Grune, Nicolás Buzeta

Pregunta 1

Dada una cadena s , encontrar la longitud de la subsecuencia palindrómica más larga en ella.

Nota 1: La subsecuencia palindrómica más larga es la subsecuencia de máxima longitud de una cadena dada que también es un palíndromo. Nota 2: Una subsecuencia es una secuencia que puede ser derivada de otra secuencia al borrar algunos o ningún elemento, sin cambiar el orden de los elementos que quedan.

- a) Piensa en un algoritmo de fuerza bruta simple. ¿Cuál es su complejidad?
- b) Piensa en un algoritmo de fuerza bruta recursivo. ¿Cuál su ecuación de recurrencia?
- c) Optimice su algoritmo recursivo para guardar los resultados recursivos.
- d) Implemente su algoritmo usando programación dinámica bottom-up, para que sea lo más eficiente posible.
- e) ¿Cuál es la complejidad de su algoritmo final?

Solución

```
a) def is_palindrome(s: str) -> bool:
    2     global n_comparisons
    3     n_comparisons += 1
    4     return s == s[::-1]
    5
    6
    7 def simple_brute_force(s: str) -> int:
    8     all_possible_subsequences: List[str] = []
    9
    10    for letter in s:
    11        previous_subsequences = all_possible_subsequences.copy()
```

```

12     previous_subsequences_with_letter = [
13         subseq + letter for subseq in previous_subsequences
14     ]
15
16     all_possible_subsequences.extend(previous_subsequences_with_letter)
17     all_possible_subsequences.append(letter)
18
19     return max(
20         (len(subseq) for subseq in all_possible_subsequences
21          if is_palindrome(subseq)),
22         default=0,
23     )

```

Con una complejidad de 2^n , ya que existen 2^n subsecuencias.

b)

```

def recursive_brute_force(i: int, j: int, s: str) -> int:
    global n_comparisons
    if i > j:
        return 0

    if i == j:
        return 1

    n_comparisons += 1
    if s[i] == s[j]:
        return 2 + recursive_brute_force(i + 1, j - 1, s)

    return max(recursive_brute_force(i, j - 1, s), recursive_brute_force(i + 1, j

```

$$LPS(i, j) = \begin{cases} 1 & \text{if } i = j \\ 2 + LPS(i + 1, j - 1) & \text{if } s[i] = s[j] \\ \max(LPS(i, j - 1), LPS(i + 1, j)) & \text{if } s[i] \neq s[j] \end{cases}$$

c)

```

def handmade_cached_recursive_brute_force_wrapper(s: str) -> int:
    n = len(s)

    cache = [[-1] * n for _ in range(n)]

    return handmade_cached_recursive_brute_force(0, len(s) - 1, s, cache)

def handmade_cached_recursive_brute_force(i: int, j: int,
s: str, cache: Any) -> int:
    if i > j:
        return 0

    global n_comparisons
    if cache[i][j] != -1:
        return cache[i][j]

```

```

17
18     if i == j:
19         cache[i][j] = 1
20         return 1
21
22     n_comparisons += 1
23     if s[i] == s[j]:
24         result = 2 +
25             handmade_cached_recursive_brute_force(i + 1, j - 1, s, cache)
26         cache[i][j] = result
27         return result
28
29     result = max(
30         handmade_cached_recursive_brute_force(i, j - 1, s, cache),
31         handmade_cached_recursive_brute_force(i + 1, j, s, cache),
32     )
33     cache[i][j] = result
34     return result

```

d)

```

def dynamic(s: str) -> int:
    2
    3     n = len(s)
    4     dp = [[0] * n for _ in range(n)]
    5
    6     for i in range(n - 1, -1, -1):
    7         for j in range(i, n):
    8             if i == j:
    9                 dp[i][j] = 1
   10                 continue
   11
   12             if s[i] == s[j]:
   13                 if i + 1 == j:
   14                     dp[i][j] = 2
   15                 else:
   16                     dp[i][j] = dp[i + 1][j - 1] + 2
   17             else:
   18                 dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
   19
   20     return dp[0][n - 1]

```

- e) Usamos un arreglo de tamaño $|s| \times |s|$ y rellenamos cada casilla 1 vez. Por lo que la complejidad final es $\mathcal{O}(|s|^2)$

Pregunta 2

Usted está por resolver un examen que tiene n preguntas. Para $1 \leq i \leq n$, la pregunta i entrega p_i puntos. Usted solo puede responder las preguntas en orden, desde la 1 a la n , sin posibilidad de retroceder. También es posible dejar ciertas preguntas sin responder. La razón por la que puede elegir hacer esto último es que algunas de las preguntas del examen son tan frustrantes que al intentar responderlas será incapaz de resolver las siguientes f_i preguntas, para $1 \leq i \leq n$. De manera formal, al resolver la pregunta i será incapaz de resolver las preguntas $i + 1, i + 2, \dots, i + f_i$. Suponga que como entrada se conocen los valores $p_1, \dots, p_n$ y $f_1, \dots, f_n$.

- Piensa en un algoritmo de fuerza bruta recursivo S para calcular la máxima nota posible. Encuentre la ecuación de recurrencia de su algoritmo.
- Asuma que tiene un arreglo $s[1..n]$. Encuentre la ecuación que da el valor de cada celda de s . Donde $s[i] = S(i)$.
- Implemente su algoritmo usando programación dinámica bottom-up, para que sea lo más eficiente posible.
- ¿Cuál es la complejidad de su algoritmo?

Solución

```
a) def recursive_brute_force(current_question: int,
2 p: List[int], f: List[int]) -> int:
3     assert len(p) == len(f)
4
5     if current_question >= len(p):
6         return 0
7
8     # Do current question
9     next_question = current_question + f[current_question] + 1
10    result_1 = p[current_question] + recursive_brute_force(next_question, p, f)
11
12    # Skip current question
13    next_question = current_question + 1
14    result_2 = recursive_brute_force(next_question, p, f)
15
16    return max(result_1, result_2)
```

$$S(i) = \begin{cases} 0 & \text{if } i \geq n \\ \max(p_i + S(i + f_i + 1), S(i + 1)) & \text{if } i < n \end{cases}$$

b)

$$s[i] = \begin{cases} p_n & i = n \\ \max(p_i + s[i + f_i + 1], s[i + 1]) & i < n \end{cases}$$

Con $s[i] = 0 \forall i > n$

Acá podemos apreciar que es la misma ecuación de la parte a, solo que ahora lo miramos como casillas de un arreglo envés de una ecuación de recurrencia.

c)

```

1 def dynamic(current_question: int, p: List[int], f: List[int]) -> int:
2     dp = [0] * len(p)
3
4     for question_index in range(len(p) - 1, -1, -1):
5         # Do current question
6         next_question = question_index + f[question_index] + 1
7         future_points = dp[next_question] if next_question < len(dp) else 0
8
9         result_1 = p[question_index] + future_points
10
11        # Don't do current question
12        next_question = question_index + 1
13        future_points = dp[next_question] if next_question < len(dp) else 0
14
15        result_2 = future_points
16
17        dp[question_index] = max(result_1, result_2)
18
19    return dp[0]
```

d) Usamos un arreglo de tamaño n y rellenamos cada casilla 1 vez. Por lo que la complejidad final es $\mathcal{O}(n)$

Pregunta 3

El Profesor Oblivious es un poco descuidado y se le han perdido los paréntesis de una expresión matemática fundamental para su investigación. Lo único que le ha quedado es una expresión matemática sin paréntesis, de la forma:

$$x_1 \diamond_1 x_2 \diamond_2 \cdots \diamond_{n-1} x_n,$$

donde $x_1, x_2, \dots, x_n$ son números enteros positivos, y $\diamond_1, \diamond_2, \dots, \diamond_{n-1}$ son operadores, cada uno de los cuales puede ser $+$ (suma) o $\times$ (multiplicación). Para poder continuar con su investigación, el Profesor se conforma con encontrar una cota superior para el valor que puede tomar su expresión.

1. Diseñe un algoritmo de fuerza bruta recursiva que permita obtener el valor máximo para una expresión dada. Puede usar una ecuación de recurrencia, pseudocódigo, o el lenguaje de su preferencia. Aún cuando hay maneras alternativas de resolver este problema, su solución tiene que ser por fuerza bruta recursiva, para poder aplicar programación dinámica en el punto siguiente.
2. Diseñe un algoritmo de programación dinámica bottom-up (es decir, un algoritmo iterativo) que permita resolver el problema de forma eficiente. Use pseudocódigo o el lenguaje de su preferencia.
3. ¿Cuál es la complejidad de su algoritmo?

Solución

```
a) def recursive_brute_force(i: int, j: int, nums: List[int]) -> int:
    2     if i == j:
    3         return nums[i]
    4
    5     possible_results: List[int] = []
    6     for k in range(i, j):
    7         left = (i, k)
    8         right = (k + 1, j)
    9         left_result = recursive_brute_force(*left, nums)
   10         right_result = recursive_brute_force(*right, nums)
   11         result = max(left_result + right_result, left_result * right_result)
   12         possible_results.append(result)
   13
   14     return max(possible_results)
```

$$V(i, j) = \begin{cases} x_i & \text{if } i = j \\ \max_{i \leq k < j} (V(i, k) \diamond_k V(k + 1, j)) & \text{if } i < j \end{cases}$$

```
b) def dynamic(nums: List[int]) -> int:
    2     n = len(nums)
```

```
3     dp = [[0] * n for _ in range(n)]
4
5     for i in range(n):
6         dp[i][i] = nums[i]
7
8     print(dp)
9
10    # Iterate all subarray lengths
11    for subarray_length in range(2, n + 1):
12        for i in range(n - subarray_length + 1):
13            j = i + subarray_length - 1
14            possible_results: List[int] = []
15            for k in range(i, j):
16                left_result = dp[i][k]
17                right_result = dp[k + 1][j]
18                result = max(left_result + right_result, left_result * right_result)
19                possible_results.append(result)
20
21            dp[i][j] = max(possible_results)
22
23    return dp[0][n - 1]
```

- c) Usamos un arreglo de tamaño $n \times n$ y rellenamos cada casilla 1 vez. En este caso nos demoramos tiempo $\mathcal{O}(n)$ en rellenar una casilla (tenemos que ver todos los k). Por lo que la complejidad final es $\mathcal{O}(n^3)$